

Evaluating a Deterministic Validator Plus Single-Iteration Repair Pipeline for LLM-Generated Network Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (N=300 paired intent→configuration tasks; pre-registration id cand-2026-001-A-prereg-v1, pre-registration SHA-256 archived) to evaluate whether a minimal guarded pipeline—single-shot LLM generation (declared model: gpt-5-mini) followed by a frozen deterministic configuration validator and at most one automated repair iteration—reduces first-pass misconfiguration relative to plain single-shot generation. Execution completed 600 episodes and used 301 model calls. Observed outcomes: baseline = 299/300 valid (99.667%); guarded = 300/300 valid (100.0%); paired counts n11=299, n10=1, n01=0, n00=0. Exact McNemar two-sided $p = 1.00$; absolute paired difference = 0.0033333; paired bootstrap 95% CI = [0.00, 0.01] (10,000 resamples, seed 1729). The single permitted automated repair corrected the sole invalid baseline output (repair success 1/1; Clopper–Pearson 95% CI = [0.025, 1.0]). The preregistered planning assumption used for power calculations (planned baseline pass rate = 0.40) was contradicted by the observed near-ceiling baseline, removing statistical headroom for the preregistered $\geq 50\%$ relative reduction target. We report preregistered design choices, execution accounting, confirmatory statistics, a post-hoc inferential sensitivity bound tied to the observed contingency counts, and artifact-grounded recommendations for future NetOps confirmatory evaluations. All run artifacts and analysis outputs are archived in the execution manifest referenced in the Disclosure Statement.

I. INTRODUCTION

Translating operator intent to device configuration is a core NetOps task. Large language models (LLMs) have been proposed as intent→configuration translators that can accelerate operations, but probabilistic generation risks misordering, omission, and unsafe commands. Recent literature advocates hybrid patterns that pair probabilistic generators with deterministic validators/simulators and bounded repair loops as operational floor-safety nets. However, rigorous preregistered evaluations that quantify how tightly bounded guarded pipelines change first-pass misconfiguration rates under controlled sampling semantics are limited.

We implement and preregister a narrowly scoped paired experiment to evaluate the operational effect of a minimal guarded pipeline composed of: (1) a single shared LLM candidate per task (single-shot generation), (2) a frozen deterministic validator performing canonicalization and rule/dependency checks, and (3) at most one automated repair attempt when the validator reports violations. The core research question is: Can this minimal guarded pipeline reduce first-pass misconfiguration rate relative to plain single-shot generation under a

controlled paired design? Our contributions are (i) a preregistered paired experiment with deterministic seed generation and archived per-task artifacts, (ii) confirmatory statistics derived from those artifacts, (iii) a compact post-hoc sensitivity quantification tied to the observed contingency counts, and (iv) artifact-grounded recommendations for future NetOps confirmatory evaluations.

II. RELATED WORK

The recent surge of LLM applications to NetOps has produced three closely related strands of work relevant to guarded generate→verify→repair architectures. First, system demonstrations and intent-to-configuration translators show that LLMs can synthesize device configuration templates from natural language intent with promising accuracy in controlled settings, but these demonstrations frequently document ordering and omission errors that threaten operational safety [10.1002/nem.2313]. Those works typically evaluate end-to-end generation quality on curated testbeds and emphasize practical engineering concerns—prompt framing, template constraints, and device-family idiosyncrasies—rather than formalized confirmatory inference. Our experiment differs by preregistering paired inferential semantics and by measuring first-pass validity under a deterministic canonicalizer/validator and bounded repair budget.

Second, adjacent domains (medical QA, code repair, and software testing) provide concrete evidence that verifier-assisted pipelines, retrieval-augmented generation (RAG), and repair loops reduce factual errors and improve grounding relative to plain generation [10.36227/techrxiv.176784505.56675782/v1, 10.2139/ssrn.6608518]. Methodologically, these works combine (a) an LLM generator, (b) an external evidence or retrieval layer to ground outputs, and (c) a verifier or test harness that identifies discrepancies which then drive targeted regeneration or repairs. Crucially, reported gains vary with verifier fidelity and retrieval quality; effectiveness in one domain does not directly imply equivalent gains in NetOps where configuration correctness depends on strict, stateful device invariants and ordering constraints. Our guarded pipeline mirrors the generate→validate→repair motif but intentionally restricts intervention to a frozen deterministic validator and a single automated repair iteration to test a minimal operational floor-guard.

Third, the network-verification literature supplies algorithmic techniques for deterministic checking of syntactic and semantic properties of configurations. Header-space and related specification-based analyses enable static reachability and policy checks on canonicalized configuration representations and have been incorporated into tools for incremental verification and real-time policy checking [10.1145/3098822.3098834]. These deterministic techniques provide the computational primitives used by many recent NetOps validators: parse to an AST/canonical form, evaluate invariant and dependency rules, and flag violations deterministically. Our validator (deterministic_netops_validator_v5) follows that pattern—canonicalizing LLM outputs to normalized ASTs, checking required command sequences and workflow policies, and producing binary pass/fail scores plus coded violations—so the present study tests the practical effect of placing that class of verifier upstream of an automated, planner-guided single repair iteration.

Finally, survey and standards-style documents highlight system and governance concerns—hallucination, overconfidence, auditability, and human-oversight erosion—that motivate guarded architectures and reproducible evaluation protocols [10.6028/nist.ai.100-2e2023]. These works argue for verifiable logs, deterministic validators, and human-in-the-loop approval when systems affect critical assets. Our preregistered design and artifact archiving respond to these calls by (a) fixing pairing semantics and repair budgets before execution, (b) deterministically generating task seeds, and (c) archiving per-task normalized configurations, validator traces, and repair prompts/responses to support reproducible inference and forensic analysis.

Taken together, prior empirical and methodological work supports three practical expectations: verifier-assisted pipelines can reduce factual/configuration errors given a sufficiently capable verifier and repair strategy; deterministic static checks are appropriate first-line validators for many configuration properties; and rigorous evaluation of floor-guard pipelines requires careful alignment of sampling semantics, task difficulty, and preregistered power assumptions. The present study contributes by executing a tightly preregistered paired test that isolates the incremental effect of a frozen deterministic validator plus a single automated repair iteration on first-pass validity, documenting where those prior expectations are met and where preregistered planning assumptions (e.g., baseline pass rate = 0.40) interact with sampling semantics to limit inferential power.

III. METHODOLOGY

Preregistration, hypotheses, and planning assumptions: The study was preregistered prior to observing outcomes (cand-2026-001-A-prereg-v1). The two preregistered confirmatory hypotheses were: H1 — the guarded pipeline (deterministic validator + ≤ 1 automated repair) reduces the first-pass misconfiguration rate by $\geq 50\%$ (relative) versus plain single-shot generation; H2 — one or fewer automated repair iterations recover $\geq 75\%$ of initially invalid configurations. The preregistration explicitly used a planning assumption of baseline pass

rate = 0.40 when computing the sample size and detectable effect: a 50% relative reduction in the preregistered framing maps the baseline pass rate 0.40 to an expected guarded pass rate ≈ 0.70 (absolute +0.30). Under conservative McNemar approximations reported in the preregistration, $N=300$ paired tasks provided $>80\%$ power to detect absolute differences on the order of 0.20 at $\alpha=0.05$; these numerical planning choices are reported here because they materially shaped the design and interpretation.

Design overview and pairing semantics: We used a paired within-task design with a deterministically generated seed list of 300 tasks. For each seed the experiment produced one sampled LLM candidate that was shared between the two conditions (shared_initial_candidate semantics): baseline (single-shot acceptance of the sampled candidate) and guarded (the same sampled candidate passed to the frozen deterministic validator and, if invalid, subjected to the preregistered automated repair adapter with at most one repair attempt). The shared-candidate pairing was chosen to isolate the incremental effect of deterministic validation and bounded automated repair on a fixed generated plan; this choice mirrors operational workflows where a single proposed plan is reviewed and possibly repaired. The tradeoff is inferential: reusing the same candidate increases within-pair correlation and reduces the number of informative discordant pairs available to McNemar-style paired tests compared with an independent-draws design. The preregistration documented this pairing decision and its expected consequences for statistical information and required discordant counts.

Task bank, inclusion/exclusion, and sampling: The task bank comprised 300 deterministic seeds produced by a frozen task generator with prespecified vendor-template constraints and intent/workflow patterns. Inclusion required that each generated seed parse to the frozen intent template; seeds that failed the deterministic parsing rule would be deterministically replaced from a preregistered reserve list (the preregistration defines replacement rules). The planned episode count was 600 (two episodes per seed), with the primary estimand computed on the set of complete pairs that satisfied the inclusion rules. The preregistered missingness and retry policy allowed a single retry for provider/API failures; the primary analysis rule ('complete_pair_primary') drops pairs only if both calls failed. These procedural choices were set before execution and are described here because they determine how failed calls are treated in the confirmatory test.

Model, generation semantics, and execution budget: The declared LLM in the execution contract was gpt-5-mini. Execution settings declared in the plan include one initial generation call per task (initial_generation_calls_per_task = 1) and a maximum of one repair attempt per task (maximum_repair_calls_per_task = 1). The preregistered execution budget and stopping rules were fixed: complete all planned pairs ($N=300$) unless technical/provider failures exhausted the allowed retries or the model-call budget. These parameters were chosen to bound the automated repair budget and to align the confirmatory estimand with an operationally plausible,

low-repair-effort floor guard.

Deterministic validator and automated repair adapter: The frozen deterministic validator (declared identifier: `deterministic_netops_validator_v5`, `validator_version`: 5.0) implements canonicalization to a normalized configuration AST, static rule and dependency checks (for example, required sequences such as shutdown→modify→restore), and a binary pass/fail decision under the preregistered scoring rule 'full intent-constraint validation'. The validator also emits structured violation codes used to classify failure categories (preregistered exploratory categories included reachability, ACL/policy, routing/forwarding, and syntax/ordering mismatches) and to steer the automated repair adapter when an attempt is permitted. The automated repair adapter is bounded: it may issue a single repair prompt transformation of the shared candidate and resubmit the transformed candidate to the validator; if the transformed candidate still fails, no additional automated attempts are made under the preregistered design. The scoring rule required that the validator's canonicalized AST satisfy the task's required command sequence and workflow constraints to be scored as a success.

Primary estimand and inferential procedures: The primary estimand is the `paired_success_rate_difference_guarded_minus_baseline` (absolute difference in per-condition success rates computed on complete pairs). The preregistered primary hypothesis test is the exact McNemar two-sided test at $\alpha=0.05$ applied to the paired pass/fail contingency table (cells `n11`, `n10`, `n01`, `n00`). We also preregistered paired bootstrap percentile 95% confidence intervals for the absolute paired difference (10,000 resamples) and prespecified sensitivity analyses: (a) treating failed model calls as failures, and (b) stratified subgroup checks by task difficulty or vendor template. Secondary estimands included repair success conditional on initial failure and per-category reductions in failure counts; secondary p-values would be adjusted by Benjamini-Hochberg FDR ($q=0.05$) when applicable. The preregistration explicitly documented that the planned baseline pass-rate assumption (0.40) drove the detectable effect calculation and that deviations from this assumption would materially change the study's statistical headroom.

Missingness, retries, and contamination controls: The preregistered missingness plan recorded every failed model call, classified failures into provider/API versus technical failures, and allowed a single retry under the adapter policy. The contamination plan required deterministic exact-and-AST equivalence checks between test-task targets and any frozen transformation/template artifacts; exact duplicates would be excluded and replaced deterministically. The primary analysis used a decontaminated paired set per these rules with prespecified sensitivity analyses including flagged/excluded sets.

Archival and scoring traceability (procedural summary): Per the preregistration and execution contract, every execution artifact needed for confirmatory inference was recorded and archived: per-task sampled candidate (`shared_initial_candidate`), validator canonicalized configu-

ration, `parsed_command_count`, `required_command_count`, violation codes, repair prompt/response when invoked, and the binary pass/fail scoring. The analysis executor uses these archived structured artifacts to construct the paired contingency table and to run the exact McNemar test and bootstrap CIs. For interpretability of the preregistered power statements we retained the original planning numerics (baseline pass rate = 0.40 → target guarded ≈ 0.70) in the public registration so readers understand the mapping between the preregistered relative effect target and the absolute differences the design sought to resolve.

IV. RESULTS

Execution accounting and artifact provenance (archived): The run completed the planned 600 episodes (300 complete pairs) without provider or infrastructure failures under the preregistered retry policy. Total `model_calls_used` recorded in the execution manifest = 301, reflecting 300 `shared_initial_generation` calls plus a single repair invocation (`stage_counts`: `shared_initial_generation`=300, `repair`=1). No deterministic exclusions for contamination were triggered by the preregistered checks; the execution and analysis artifacts enumerating per-task normalized responses, validator traces, and repair traces are archived and are the primary source for the numerical counts below.

Primary, pair-level outcomes (artifact counts): From the archived paired contingency artifact, the per-condition success tallies are: `baseline_success_count` = 299/300 (99.6667%), `guarded_success_count` = 300/300 (100.0%). The full paired contingency table derived from the archived analysis artifact is `n11` = 299 (both arms success), `n10` = 1 (guarded success only), `n01` = 0 (baseline success only), `n00` = 0 (both fail). These contingency counts are reproduced verbatim in the archived artifact `analysis/paired_contingency_table.csv` and underpin all confirmatory inference reported here.

Confirmatory inferential results (preregistered analyses using archived artifacts): Applying the preregistered exact McNemar two-sided test to the observed contingency yields $p = 1.00$. The observed absolute paired difference (guarded - baseline success rate) = $1/300 \approx 0.003333$. Paired bootstrap percentile 95% CI for the absolute paired difference computed per the preregistered procedure (10,000 resamples, `bootstrap_seed` = 1729) = [0.00, 0.01]. Repair outcomes: the single allowed automated repair corrected the sole invalid baseline output (repair success 1/1). The preregistered exact Clopper-Pearson 95% interval for that conditional repair success is [0.025, 1.0], reflecting the wide uncertainty when only a single initial failure was observed.

Post-hoc sensitivity and interpretive bounds grounded in observed discordant counts: The total number of discordant pairs observed is `n10` + `n01` = 1. Under the paired within-task sampling semantics used, this discordant total imposes a strict upper bound on the resolvable absolute paired difference equal to $(n10 - n01)/N = 1/300 \approx 0.00333$. Practically, that bound means the empirical data cannot support detection of large absolute improvements (for example, the preregistered $\geq 50\%$

relative reduction target, which was premised on an assumed baseline pass rate of 0.40 and corresponds to an absolute increase of ≈ 0.30) because achieving such an effect with $N=300$ would have required on the order of many tens of discordant pairs (the preregistration’s heuristic estimate suggested ≈ 90 discordant pairs would be needed to observe such an effect size under conservative discordant assumptions). The archived contingency artifact and the bootstrap CI concretely illustrate this limitation: the 95% CI for the absolute paired difference terminates at 0.01, well below the preregistered planning effect.

Why the exact McNemar $p = 1.00$ arises here (artifact-grounded intuition): Exact McNemar inference bases significance on the imbalance of the two discordant cell counts (n_{10} versus n_{01}). With only one discordant pair (1 vs 0) the exact two-sided test computes a tail probability that includes that single outcome and any outcomes at least as extreme under the null; with such minimal discordance the resulting two-sided p is not small and in this case equals 1.00 as reported by the preregistered executor. The archived `paired_contingency_table.csv` and `analysis/analysis_log.jsonl` record the discordant counts and the exact test invocation that produced this p -value.

Diagnostic traceability and representative artifact observations: For every episode the deterministic validator trace archived fields including `normalized_configuration`, `parsed_command_count`, `required_command_count`, `violation_codes`, and `repair_applied_flags`. The single baseline failure is fully traceable in the archived task artifact (`execution/scoring/task-000XXX-baseline.json` entry) and shows the validator-reported violation that triggered the one repair invocation; the archived repair prompt/response pair documents the change applied and the validator’s post-repair canonicalized configuration which satisfied the full intent constraints. These per-task artifacts are the authoritative evidence for the repair outcome and are enumerated in the execution manifest and representative task listings.

Implications of `shared_initial_candidate` pairing for observed information: The preregistered shared-candidate design intentionally reused the same generated plan across both arms to isolate the validator/repair effect on a fixed candidate. While operationally motivated, this design choice increases within-pair correlation and consequently reduces expected discordant counts compared with an independent-draws design; fewer discordant pairs directly reduce the statistical information available to McNemar inference for a given N . The observed near-ceiling baseline (299/300) in combination with the shared-candidate pairing therefore explains the very small observed discordant total and the resulting inability to confirm the preregistered large relative effect. These relationships between pairing semantics, baseline prevalence, and discordant counts are visible in the archived `deterministic_reconciliation.json` and `analysis/condition_summary.csv` artifacts and informed the post-hoc sensitivity statements above.

V. DISCUSSION

The guarded pipeline demonstrably remediated the single invalid baseline output observed in this run: the deterministic validator flagged the violation and the single permitted automated repair produced a canonicalized configuration that the validator accepted. This outcome empirically supports the practical utility of a frozen validator plus a tightly bounded repair step as a floor-safety mechanism that can convert some validator-detected failures into valid first-pass configurations.

However, the run also illustrates an important inferential and design caveat: confirmatory power for paired tests depends critically on the number of informative discordant pairs. Our preregistered `shared_initial_candidate` pairing was chosen to model operational workflows that examine and remediate a single generated plan. That design reduces within-pair heterogeneity and isolates the effect of validation+repair on a given candidate, but it also reduces the expected discordant pair count relative to independent draws per arm. In our execution the baseline pass rate was 299/300, producing only one discordant pair and thereby imposing a strict numerical ceiling on resolvable absolute improvement ($1/300 \approx 0.00333$). Consequently, the preregistered $\geq 50\%$ relative-reduction target—formulated under a planning assumption `baseline_pass_rate = 0.40`—became unattainable given the observed prevalences. The archived preregistration explicitly records the `baseline_pass_rate = 0.40` planning assumption and the analysis artifacts show the realized counts; together they explain why the confirmatory hypothesis could not be supported despite the guarded pipeline correcting the single failure it encountered.

Operationally useful lessons flow directly from these artifact-grounded facts. First, when baseline error prevalence is uncertain, researchers should design task banks or sampling procedures to produce an expected number of initial failures (or discordant pairs) aligned with the planned detectable effect. For the preregistered aim in this study, the analysis artifacts indicate that on the order of 90 discordant pairs would have been needed to resolve an absolute change comparable with the preregistered target (≈ 0.30 absolute). In practice, investigators can (a) increase task difficulty, (b) include adversarial or stress-test augmentations, or (c) adopt stratified sampling by difficulty to concentrate statistical information where it is most informative. These are operational design options rather than claims about guaranteed outcomes; their selection should be preregistered and justified against expected discordant counts.

Second, choice of pairing semantics must be matched to the operational claim. If the goal is to measure whether validator+repair improves the probability that an operator will accept a generated plan when the generator is sampled afresh per condition, independent per-arm draws increase discordance and statistical information for McNemar-style inference. If, instead, the workflow examines and remediates a single proposed plan (the `shared_initial_candidate` semantics used here), then evaluation budgets should anticipate reduced discordant counts and compensate via deliberately higher-difficulty tasks or larger N . The preregistration tradeoff—higher within-pair

control versus greater discordant information—should be made explicit in power computations; our archived power plan assumed `baseline_pass_rate = 0.40` and thus did not account for the realized near-ceiling baseline.

Third, repair-effect estimands are inherently conditional on initial failures and require sufficient counts of those failures to estimate repair effectiveness precisely. In this run the observed repair success was 1/1, which is artifact-grounded but statistically imprecise (Clopper–Pearson 95% CI = [0.025, 1.0]). When repair effectiveness is a primary operational claim, studies should be powered to observe an adequate number of initial failures so that conditional repair success has narrow confidence intervals; appropriate sample sizing depends on the target precision and should be preregistered. The deterministic artifacts archived here (per-task validator traces and repair prompts) exemplify the level of per-case traceability required to support conditional estimands and subsequent diagnostics.

Finally, construct and external validity remain constrained in this study. Our validator performed deterministic static canonicalization and rule checks rather than full control-plane or packet-level emulation; therefore, runtime semantics and emergent control-plane interactions were not exercised. The task bank is synthetic and vendor-template coverage was constrained, and only one declared model (gpt-5-mini) and one validator implementation were used. These limitations narrow direct production generalization but do not diminish the methodological point: artifact-grounded preregistration combined with deterministic validator traces yields reproducible counts that clarify what was actually tested and which inferential questions remain open.

In sum, the empirical artifact shows that small guarded pipelines can correct individual validator-detected errors, but proving large relative reductions in misconfiguration rate under paired designs requires aligning task difficulty, pairing semantics, and sample size with the planned estimand. We therefore recommend that future confirmatory NetOps evaluations (a) preregister pairing semantics and the expected discordant-pair count implied by the power calculation, (b) calibrate task difficulty or use adversarial augmentation to achieve a target initial-failure prevalence when necessary, and (c) archive per-task validator and repair traces to support transparent conditional estimands and post-hoc diagnostics. The archived execution and analysis artifacts accompanying this study provide a concrete example of how those recommendations can be operationalized.

VI. LIMITATIONS

- Synthetic task bank and constrained vendor-template scope limit external validity to broad production environments.
- Single LLM (gpt-5-mini) and a single validator implementation restrict generality across model families and validator designs.
- The preregistered power plan assumed a baseline pass rate = 0.40; the observed baseline (299/300) contradicts

that assumption and removed headroom to detect the preregistered $\geq 50\%$ relative reduction.

- Sampling semantics (shared_initial_candidate) reused the same initial candidate across paired arms, increasing within-pair correlation and reducing discordant pairs available to McNemar inference.
- Validation used deterministic configuration/constraint checks rather than full dynamic emulation of runtime semantic effects; actual runtime consequences of configurations were not executed and remain unmeasured.
- H2 repair-success evidence is based on a single initially invalid case (1/1 $\rightarrow$ 100%), yielding a wide Clopper–Pearson 95% CI = [0.025, 1.0]; larger counts of initial failures are required to estimate repair effectiveness precisely.

VII. CONCLUSION

We executed a preregistered paired experiment (N=300 pairs) comparing single-shot LLM generation (gpt-5-mini) to a guarded pipeline consisting of a frozen deterministic validator and a single automated repair iteration. Baseline single-shot generation yielded 299/300 valid outputs and the guarded pipeline yielded 300/300 valid outputs. The single automated repair corrected the lone invalid baseline output, but the observed near-ceiling baseline contradicted the preregistered planning assumption (baseline pass rate = 0.40) and removed statistical headroom to detect the preregistered $\geq 50\%$ relative reduction. Future confirmatory NetOps evaluations should (a) calibrate task difficulty to produce sufficient initial failures or target discordant counts, (b) preregister pairing semantics and repair budgets explicitly, and (c) include emulation to measure runtime semantic effects when operational deployment claims are intended. All raw outputs, validator traces, repair traces, the execution manifest, and analysis artifacts are archived and enumerated in the Disclosure Statement to permit reproduction of the reported counts and intervals.

DISCLOSURE STATEMENT

This study was executed autonomously after the autonomy lock under the archived master prompt (provenance/master_prompt.txt; SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the master prompt prior to the autonomy lock; no manual human editing of manuscript technical content, figures, tables, or references occurred after the lock. Preregistration: cand-2026-001-A-prereg-v1 (the preregistration record was created before observing final outcomes; preregistration SHA-256 is recorded in the execution manifest). Declared model and tooling: gpt-5-mini (declared_model_version), adapter family hosted_netops_gvr_v1, frozen deterministic validator/repair adapter deterministic_netops_validator_v5. Execution accounting (archived): planned episodes = 600, completed episodes = 600, complete pairs = 300, model_calls_used = 301 (300 shared initial generation

calls + 1 repair invocation), repair-stage invocations = 1. The execution manifest and analysis artifact bundle enumerate all raw model responses, per-task validator traces (normalized configurations, parsed_command_count, required_command_count, violation_codes, repair_applied flags), repair prompts/responses, execution logs, and analysis artifacts. The archived execution manifest and analysis bundle are the authoritative source for the numeric counts reported in this manuscript. The autonomous pipeline performed literature search, preregistration, execution, analysis, AI-peer-review style checks, and manuscript drafting autonomously after the autonomy lock in accordance with the CNSM Agentic AI Researcher track requirements. The preregistered planning assumption used in power calculations (planned baseline pass rate = 0.40) is explicitly reported here because it materially affects interpretation of the confirmatory result.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Oghenekeno Hilkhiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1
- [3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518
- [4] Goutam Adwant, Manjari Srivastav, "Evidence Coverage Evaluator: A Comprehensive Framework for Assessing Grounding Quality in Retrieval-Augmented Generation Systems", 2026, 10.36227/techrxiv.176784505.56675782/v1
- [5] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [6] Apostol Vassilev, Alina Oprea, Alie Jean Fordyce, Hyrum S. Anderson, "Adversarial machine learning :", 2024, 10.6028/nist.ai.100-2e2023